SRS: Scope Change Desk (C35)

Oct 17, 2026 · @krishna

Purpose and scope

Scope Change Desk turns a signed Statement of Work (SOW) and a later, amended SOW into a priced, approved change order, so extra work is never done unpaid or unrecorded.

- **Problem:** clients ask for more work in vague terms. Freelancers and small agencies either absorb it (scope creep) or argue about price after the fact.
- **Product:** a client posts a job, a freelancer applies and is accepted, and they talk in a shared project workspace. Then the signed SOW is uploaded as the baseline, the amended SOW is uploaded when the client wants more, and the site lists what was added, prices each addition from the agreed rate and hours, flags anything unusual, and records the client's approval or rejection.
- **USP:** the change-order step is clear, explainable and recorded. Every price shows its hours, rate and assumptions, and every flag shows why it fired.
- **Positioning:** a basic job board (post, apply, accept, chat) in the style of Fiverr and Zoho, with the change-order desk as the feature that sets it apart. Advanced marketplace features are out of scope, and nothing here is legal advice.
- **Context:** Fynd x NMIMS Hyderabad 24-hour hackathon, 17-18 October 2026, case C35. All data is synthetic. We never claim real revenue, real demand or production readiness.

Users and roles

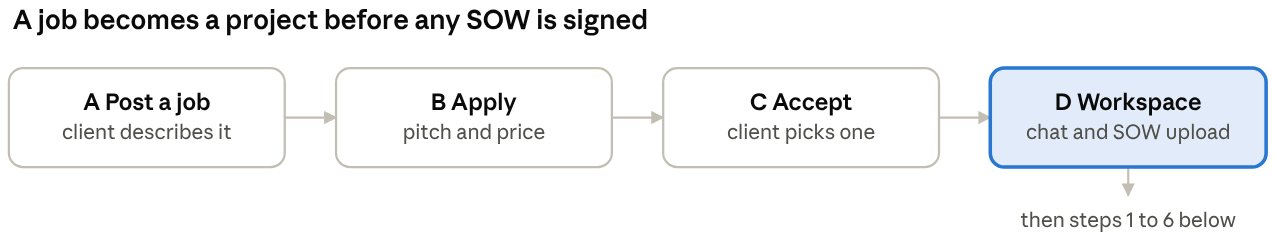
There are two roles, each with its own dashboard, and either one may upload the signed SOW.

Role	Who	Can do	Cannot do
Freelancer	Freelancer or small agency account lead	Browse open jobs and apply, chat in the project workspace, upload the signed SOW and amendments, correct extracted items, raise requests, see capacity, flags and approval status, reopen a rejected change with a reason	Approve their own change order
Client	The paying side, one named approver per project	Post jobs, review applicants and accept one, chat in the project workspace, upload an amendment, see each priced change, approve or reject with a reason	Edit the signed baseline or the rate

A demo role switch ("Continue as Client" or "Continue as Freelancer") comes first. Real sign-in through Supabase Auth is added only if time remains after the feature freeze.

User journey

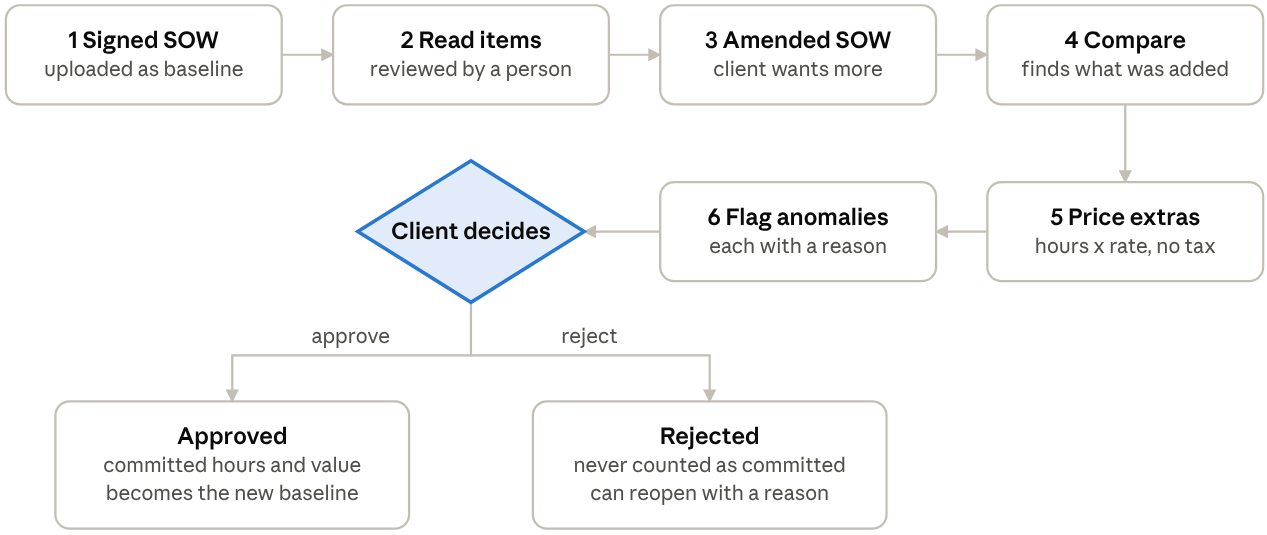
A client posts a job and hires a freelancer in a shared workspace, then every extra request after the signed SOW is compared, priced and decided.



hiring · 4 steps

Steps A to D hire the freelancer and open the project workspace. Steps 1 to 6 then control every change.

Extra work is priced and approved before it is booked



core journey · 6 steps, 1 decision

A rejected change is recorded but never counted as committed. An approved one becomes the baseline for the next comparison.

Functional requirements

There are 30 requirements: 21 are Must, 8 are Should and 1 is Could. 12 are already built in the local prototype, which has a plain UI, no roles, no job board and no SOW reader yet.

ID	Requirement	Priority	Status	Judging link
FR1	Upload a signed SOW (PDF, .txt, .md) as the project baseline	Must	To build	Working journey
FR2	Extract deliverables, included, excluded, rate, hours and deadline from an SOW	Must	To build	Working journey
FR3	Let the user review and correct extracted items before comparing	Must	To build	Reliability
FR4	Upload an amended SOW as a new version of the same project	Must	To build	Working journey

ID	Requirement	Priority	Status	Judging link
FR5	Compare two SOWs and label each item new, previously excluded, changed, removed or unchanged	Must	To build	Working journey
FR6	Price each extra item as hours x rate, before tax, with assumptions shown	Must	Built	Check 1
FR7	Keep unknown hours as null and ask for clarification, never price at zero	Must	Built	Reliability
FR8	Client approves or rejects each change with a reason; freelancer sees the status	Must	Built (no roles)	Check 2
FR9	A rejected change is never committed; it can only return through a reopen with a reason	Must	Built	Check 2
FR10	Re-uploading the same content is ignored and logged, with no double charge or double booking	Must	Built	Check 3
FR11	Refuse approval when the hours exceed remaining capacity	Must	Built	Reliability
FR12	Keep an audit trail with a sequence number and version for every action	Must	Built	Handoff
FR13	Two dashboards, one for the client and one for the freelancer	Must	To build	Working journey
FR14	Show anomaly flags on each change, each with its reason	Should	To build	Improvement
FR15	Apply a rush multiplier to urgent requests, labelled as an assumption	Should	Built	Edge case
FR16	Log the studio's own defect rework as free but capacity-consuming	Should	Built	Edge case
FR17	Export the saved record as JSON and CSV	Should	Built	Handoff

ID	Requirement	Priority	Status	Judging link
FR18	Save to a database and serve from a live link	Must	To build	Handoff
FR19	An approved amendment becomes the new baseline for the next comparison	Should	To build	Working journey
FR20	Optional AI reader for messy SOWs, with a labelled manual fallback	Could	To build	Optional
FR21	A demo panel that runs the three case checks in printed order	Must	Built	Case checks
FR22	Clear messages for bad files, bad JSON and negative hours, with no crashes	Must	Built	Reliability
FR23	Client posts a job with title, description, budget and deadline	Must	To build	Working journey
FR24	Freelancers browse open jobs	Must	To build	Working journey
FR25	Freelancer applies with a short pitch and a proposed price	Must	To build	Working journey
FR26	Client accepts one applicant, which creates the project and closes the job to new applicants	Must	To build	Working journey
FR27	Project workspace where the client and freelancer exchange messages and upload the SOW	Must	To build	Working journey
FR28	Job status of open, in progress or closed	Should	To build	Working journey
FR29	Basic profile with name, role and skills	Should	To build	Working journey
FR30	Project overview showing SOW versions, change orders, and hours used against capacity	Should	To build	Working journey

Business rules

Every rule below lives in one tested module, so the same numbers appear in the UI, the exports and the demo.

Rule	Behaviour
Price	Amount = hours x rate x multiplier, rounded to whole rupees, shown before tax
Tax	Never mixed into the change amount. GST of 18% for Indian clients is shown on a separate line (per the example SOWs)
Unknown	A null value means unknown, never zero. The item has no price and asks for clarification
Rate source	The signed SOW's rate. If the SOW has none, the team's illustrative rate card, labelled as an assumption
Hours source	The amended SOW's stated hours. If missing, a typical value from the SOW library, labelled as an assumption. If neither exists, unknown
Change versus revision	Work inside the included revision rounds is free. A new deliverable, a changed brief or extra rounds is a change
Excluded item	An item the signed SOW lists as excluded is always a paid change
Own defect	The studio's own error is fixed free, still uses capacity, and is logged as warranty work
Unclear request	A request with no measurable outcome ("make it pop") gets clarifying questions, not an estimate
Rush	Delivery inside 48 hours of the request multiplies the change by 1.5. This is an assumption and is labelled as one
Approval	One named approver per project. An approval applies only to the version it names; a newer version needs a fresh approval

Rule	Behaviour
Capacity	Committed hours = approved + own-defect + in-scope. Approval is refused when hours exceed what remains
Idempotent import	Same id and same content hash is ignored. Changed content creates a new version and voids the old approval
Hiring	One accepted freelancer per job. Accepting closes the job to new applications and creates the project workspace. Messages and SOW versions stay with the project record

Anomaly flags

Anomaly detection is a set of explainable rules, not a trained model. Each flag names its trigger, so a person can check it. The data is synthetic, so we label it "rule-based now; a learned model needs real data".

Flag	Trigger	Message shown
Previously excluded	New item matches an item in the signed SOW's Excluded list	This was excluded in the signed SOW, so it is a paid change
Hours outlier	Hours are above 2x or below 0.5x the typical value for that kind of work (threshold to tune)	Hours are far from the usual range for this work
Scope up, price flat	Deliverables are added but the total price is the same or lower	More work for the same price, so check the price
Compressed deadline	Amended deadline is shorter than the signed one	Shorter deadline, so the rush rule may apply
Duplicate	Same item already exists as an approved change	Already approved, will not be charged again
Over capacity	Item hours exceed remaining capacity	Needs more hours than remain, so approval is blocked

Flag	Trigger	Message shown
Unknown hours	Hours are null	Hours unknown, so no price until clarified
Conflicting approval	A second person approves or rejects an already-decided change	Already decided by another approver

Acceptance checks

The three case outcome checks are worth 25 of 100 engineering points (10 + 10 + 5) and must be shown live in this order. Each is already an automated test and a button in the demo panel.

1. **Estimate (10 points):** six extra hours at ₹1,000 per hour shows a change estimate of ₹6,000, before taxes.
2. **Rejection (10 points):** a rejected change does not silently become committed work. Committed value stays ₹0, approving it again is blocked, and re-importing it does not revive it.
3. **Repeat import (5 points):** importing the same request again does not double-charge or double-book. Value, hours and request count stay unchanged, and the duplicate is logged.

Failure cases that must show a clear message and never crash:

Case	Expected result
Uploaded file is not a PDF, .txt or .md, or is too large	Refused with a message naming the allowed types and size
PDF has no readable text (scanned image)	Message says the text could not be read and offers manual entry
JSON is malformed	Message says it is not valid JSON, and nothing is imported
Hours are negative or not a number	Message names the request and the problem, and the whole batch is refused
Request is unclear	Marked needs clarification, with no estimate

Case	Expected result
Urgent deadline	Rush multiplier applied and labelled as an assumption
Own defect	Booked as capacity, charged ₹0
Conflicting approvals	Second decision refused, first decision kept

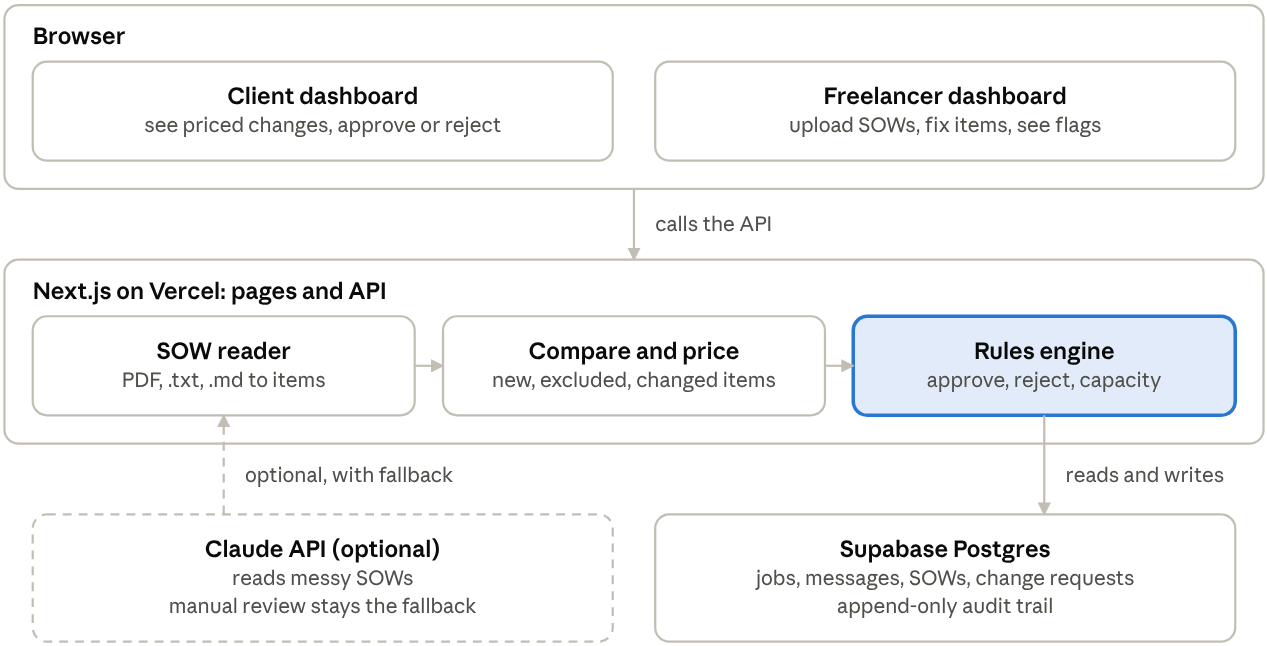
Non-functional requirements

- **Reliability:** every input is validated, a batch with one bad row imports nothing, and no user action may crash the page. Errors show as messages that say what to fix.
- **Security:** keys and the Supabase service key live only in Vercel environment settings, never in the repo or in chat. Role checks run on the server, not in the browser. Uploaded text is shown as plain text and never executed. File type and size are limited.
- **Public repo:** the repository is public, so no secrets, no real client names and no private data in any file or SOW.
- **Data honesty:** all data is synthetic and labelled so. Estimates built from assumptions say so on screen.
- **Explainability:** every price shows hours, rate and multiplier. Every flag shows its reason.
- **Reproducibility:** one command to install and run, one command to test, and a README that matches the code.
- **Performance target:** comparing two SOWs of about 20 pages takes under 5 seconds (target to measure).
- **Concurrency:** two people acting at once must not double-book. A version check rejects a stale write.

Data model and tech stack

The business rules stay in one plain JavaScript module that already has 21 passing tests. The web layer and the database wrap it, so a UI change cannot change the money logic.

The rules engine sits between the pages and the database



architecture · 3 layers, 1 optional service

The pages call the API, the API runs the reader, the comparison and the rules engine, and the rules engine saves every decision to the database.

Layer	Choice	Why we chose it (answer for the judges)
Rules engine	Plain JavaScript module, tested with Node's built-in test runner	Every number is explainable and testable. No dependencies, so it runs anywhere
Web app and API	Next.js (React) on Vercel	One project for both dashboards and the API, and one deploy for a live link
Database	Supabase Postgres	Shared state for two browsers, a unique key on request id and content hash blocks duplicate charges at the data layer, and an append-only audit table

Layer	Choice	Why we chose it (answer for the judges)
SOW reading	PDF text extraction library plus a section parser	Our example SOWs use consistent headings, so rules are reliable. A person reviews the result
Optional AI reader	Claude API with a manual fallback	Handles messy SOWs. The fallback is always labelled, as the brief requires

Main tables:

Table	Key fields
projects	id, job, name, rate per hour, currency, available hours, current version
sow_documents	id, project, version, kind (baseline or amendment), file name, text, content hash, uploaded by
sow_items	id, document, section (deliverable, included, excluded), text, hours, deadline
change_requests	id, project, item, version, state, hours, amount, flags, decided by, reason
audit	sequence, time, action, request, version, actor, detail (append-only)
users	id, role (client or freelancer), name, skills
jobs	id, client, title, description, budget, deadline, status
applications	id, job, freelancer, pitch, proposed price, status
messages	id, project, sender, body, time

Out of scope

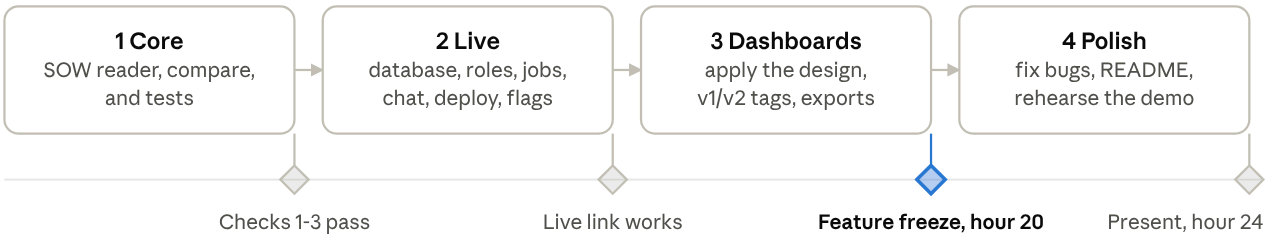
These are deliberately left out of the 24-hour build. Advanced marketplace features belong on the roadmap slide.

- Advanced marketplace features: recommendations and ranking, reviews and ratings, escrow, disputes and video calls.
- Real payments. Any charge is a priced change order and an approval record. A payment screen, if shown, runs in a labelled test mode.
- Legal advice or any claim that a generated record is a binding contract.
- E-signature of the SOW.
- A trained machine-learning model. Flags are explainable rules.
- Reading scanned PDFs by image recognition.
- Multiple currencies and filing real taxes.

Build plan

The build runs in four phases, with the three case checks passing before anything else is polished and a feature freeze at hour 20.

Four phases lead to a feature freeze at hour 20



build plan · 4 phases, 4 gates

Bands are in order, not to scale. Hour marks come from the event brief; the rest is a proposal to adjust against the real clock.

Who	Work
Krishna with Claude	Jobs, applications and chat, SOW reader, compare, rules, anomaly flags, database, deploy, v1 and v2 tags
Sanjana	Frontend design folder for the client and freelancer dashboards

Who	Work
Harini	Complete her SOWs with rate, hours, exclusions and a change-request bank, in `docs/sow/` on her branch
Swapnil	Fix his SOW upload: right folder, his own five missing roles, pricing content

Risks and open questions

The biggest risks are SOW files that arrive in different formats and a free-tier server that sleeps before the demo.

Risk	Impact	Mitigation
Team SOWs use different formats (Harini's lack exclusions, rates and hours; Swapnil's PDF has no pricing content)	Comparison and pricing cannot read them	Fix them to Sanjana's format, and keep a review screen where a person corrects extracted items
PDF text extraction fails or is wrong	Wrong items compared	Review step, plain-text upload as a second route, labelled manual entry as a fallback
Fixture was reconstructed, not the official file	A judge's data may differ	Swap in the official fixture.json if obtained; the engine reads it unchanged
Free-tier hosting sleeps after idle	First load takes 30 to 60 seconds in the demo	Open the live link several minutes before presenting
Two people act at once	Possible double booking	Version check on the desk plus a unique key on request id and content hash

Risk	Impact	Mitigation
Marketplace features crowd out the case checks	The case checks are not finished	Keep the marketplace basic (post, apply, accept, chat), build it after the three checks pass, and cut it first if time runs short
Design from Sanjana arrives late	UI work compressed	Plain role dashboards ship first; the design is applied on top

Open questions:

- [ ] What does the business judging rubric reward? (Not yet known: ask the organisers.)
- [ ] Who on the team owns the MBA-side work: the per-seat price test, first-customer path and the three replayed scope requests?
- [ ] Is a labelled test-mode payment screen wanted, or is the approved change order the end of the flow?
- [ ] Which PDF text library do we use? To be decided when the reader is built.
- [ ] Is the official fixture.json available from the organisers?